



The Lazy Developer

17

MODULE

Other Add-Ons & Technology Partners

Explore additional add-ons, plugins, and technology partnerships to further enhance your web app's capabilities.

Table of Contents

- 01. Enhancing Your Web App (20 min)
- 02. User flow analysis. (22 min)
- 03. Crawling other websites. (22 min)
- 04. Code validation to ensure your code works well. (22 min)
- 05. Performance analysis. (24 min)

Enhancing Your Web App with Add-Ons, Plugins & Technology Partners

"Think of your web app like a smartphone fresh out of the box. Sure, it makes calls and sends texts, but the real magic happens when you start installing apps. Weather, banking, fitness tracking, a spirit level you'll use exactly once... each app extends what your phone can do without you having to build those features from scratch. Add-ons and integrations work the same way for your web app."

You've built your app. It works. Users can sign up, log in, and do the core thing your app was designed to do. But now you're starting to think about the extras. How do I know if people are actually using my app? What happens when something breaks at 2 AM? How do I make my images load faster? Can I add live chat without building a whole messaging system?

This is where the add-on and plugin ecosystem comes in, and it's genuinely one of the most exciting parts of building a modern web app. Instead of spending weeks building an analytics dashboard from scratch, you can drop in a tool that's been refined by thousands of developers over years. Let's explore what's available and how to make smart decisions about what to add.

Why Add-Ons Matter (And Why You Shouldn't Build Everything Yourself)

There's a temptation, especially when you're learning to build with AI, to think, "I could just ask Cursor to build that for me." And sometimes that's the right call. But here's the thing: some problems have been solved really well by dedicated companies with entire teams focused on nothing but that one problem.

Error tracking is a perfect example. Could you build a basic error logger? Sure. But Sentry has spent over a decade building a system that catches errors, groups them intelligently, shows you the exact line of code that failed, tells you which users were affected, and alerts you on Slack. Rebuilding that would take months.

The golden rule: Build what makes your app unique. Buy (or integrate) everything else.

Here's a quick decision framework:

Question	If Yes...	If No...
Is this your app's core value?	Build it yourself	Consider an add-on
Does a mature, affordable tool exist?	Use the tool	Build it or find an alternative
Will maintaining this distract from your product?	Use a tool	Build it if it's simple
Do you need it working by next week?	Use a tool	You might have time to build
Is data privacy a critical concern?	Evaluate carefully	Standard tools are usually fine

The Major Add-On Categories

Let's walk through the main categories of tools you'll likely want to add to your app at some point. Think of this as a menu, not a shopping list. You don't need all of these on day one.

Analytics: Understanding Your Users

Analytics tools help you answer the fundamental question: "What are people actually doing in my app?" Without analytics, you're flying blind.

Tool	Best For	Free Tier	Integration Style
Google Analytics (GA4)	General website traffic, SEO insights	Very generous	Script tag
Vercel Analytics	Performance metrics for Vercel-hosted apps	Basic tier free	npm package
PostHog	Product analytics, feature flags, session replay	1M events/mo free	npm package or script
Hotjar	Heatmaps, session recordings, user feedback	Basic free plan	Script tag
Mixpanel	Event-based product analytics, funnels	20M events/mo free	npm package or API

For most apps just starting out, Google Analytics plus one product analytics tool (PostHog or Hotjar) is a solid combination. Google Analytics tells you about traffic and acquisition (where users come from), while product analytics tells you what they do once they're inside your app.

Prompt for Cursor: "Help me add Google Analytics 4 to my React + Vite app. I need it to track page views automatically when the route changes, and I want to set up a custom event for when a user completes signup."

Error Tracking: Catching Problems Before Users Report Them

Here's a harsh truth: users rarely report bugs. They just leave. Error tracking tools catch exceptions, crashes, and unexpected behavior automatically, then alert you so you can fix things before your inbox fills with complaints.

Tool	Best For	Free Tier	Key Feature
Sentry	Full-stack error tracking	5K errors/mo	Stack traces with source maps
LogRocket	Session replay with error context	1K sessions/mo	Video replay of user sessions
Bugsnag	Error monitoring with release tracking	7.5K events/mo	Release health dashboard

Sentry is the industry standard for good reason. It integrates with both React (frontend) and Express (backend), giving you a complete picture when something breaks.

Prompt for Cursor: "Help me add Sentry error tracking to my React + Express app. Set it up on both the frontend and backend, configure source maps for readable stack traces, and add a test button that triggers a sample error so I can verify it's working."

Performance: Making Your App Fast

Speed matters more than you think. Google's research shows that 53% of mobile users abandon a site that takes more than 3 seconds to load. Performance tools help you identify and fix bottlenecks.

Tool	What It Does	Integration Style
Cloudflare	CDN, DDoS protection, DNS management	DNS configuration
Cloudinary	Image optimization, transformation, CDN delivery	API / SDK
imgix	Real-time image processing and optimization	URL-based API
Vercel Speed Insights	Core Web Vitals monitoring	npm package

Cloudflare is practically a must-have for any production app. The free tier gives you a global CDN (Content Delivery Network, meaning your app loads from servers close to your users), DDoS protection, and SSL certificates. It's one of the best free tools in the entire web ecosystem.

For images specifically, Cloudinary is fantastic. Instead of manually resizing and compressing every image, you upload once and Cloudinary serves the perfect size and format automatically based on the user's device.

Communication: Talking to Your Users

Live chat widgets, chatbots, and support tools let you communicate with users directly inside your app.

Tool	Best For	Free Tier	Notable Feature
Tawk.to	Live chat	Completely free	Unlimited agents
Crisp	Chat + shared inbox	2 agents free	Built-in chatbot
Intercom	Full customer platform	No free tier (\$\$\$)	AI chatbot, help center

If you're just starting out and want live chat, Tawk.to is hard to beat because it's entirely free with no agent limits. Crisp is excellent if you want something a bit more polished with a shared inbox for team collaboration. Intercom is the Rolls-Royce of customer communication, but it's priced for funded startups and enterprises.

Prompt for Cursor: "Add the Tawk.to live chat widget to my React app. I want it to appear on every page except the login and signup pages. Here's my Tawk.to property ID: [YOUR_ID]."

Payments Beyond Stripe

We've covered Stripe in other modules, but it's worth knowing there are alternatives depending on your use case.

Tool	Best For	Fees	Key Advantage
Stripe	Most SaaS apps, subscriptions	2.9% + 30c	Best API and docs

PayPal	International payments, older demographics	2.9% + 30c	Brand recognition
LemonSqueezy	Digital products, handling global tax	5% + 50c	Merchant of record (handles VAT/sales tax)
Paddle	SaaS subscriptions with tax compliance	5% + 50c	Merchant of record

LemonSqueezy and Paddle are especially interesting because they act as your "merchant of record." This means they handle all the complicated global tax stuff, VAT collection, invoicing, and compliance. If you're selling to customers worldwide and don't want to deal with tax laws in 50 different countries, these are worth the higher fees.

Email Services

Sending emails from your app (welcome emails, password resets, notifications, marketing) requires a dedicated email service.

Tool	Best For	Free Tier	Integration
Brevo (formerly Sendinblue)	Transactional + marketing emails	300 emails/day	SDK / API
SendGrid	High-volume	100 emails/day	API
Resend	transactional email Developer-friendly email with React templates	3K emails/mo	API / SDK
Mailgun	Transactional email, great deliverability	5K emails/mo for 3 months	API

Brevo is a great all-rounder because it handles both transactional emails (password resets, order confirmations) and marketing emails (newsletters, promotions) from one platform. Resend is newer but has a brilliant developer experience, especially if you're using React, because you can write your email templates as React components.

How Add-Ons Actually Connect to Your App

There are four main ways tools integrate with your app. Understanding these patterns helps you evaluate new tools and know what to expect:

1. Script Tag (Easiest)

You paste a small snippet of JavaScript into your HTML. Google Analytics and most chat widgets work this way. The downside is you have less control and it can affect page load speed.

```
<!-- Example: Adding a script tag -->
<script async src="https://www.googletagmanager.com/gtag/js?id=G-XXXXXXXXXX"></script>
```

2. npm Package (Most Common for React Apps)

You install a library and import it into your code. This gives you full control and TypeScript support. Sentry, PostHog, and Vercel Analytics work this way.

```
npm install @sentry/react
```

3. REST API (Backend Integration)

Your server makes HTTP requests to the tool's API. Brevo email sending, Stripe payments, and most backend integrations work this way.

```
// Example: Calling an API from your backend
const response = await fetch('https://api.brevo.com/v3/smtplib/email', {
  method: 'POST',
  headers: { 'api-key': process.env.BREVO_API_KEY },
  body: JSON.stringify(emailData)
});
```

4. SDK (Software Development Kit)

A more comprehensive library that wraps API calls in convenient functions. Supabase, Firebase, and Stripe all provide SDKs.

```
// Example: Using Supabase SDK
import { createClient } from '@supabase/supabase-js';
const supabase = createClient(url, key);
```

A Real-World Integration Walkthrough

Let's say you've built a recipe-sharing app and you want to add three things: error tracking, basic analytics, and email notifications when someone comments on a recipe.

Here's how you'd approach it with AI:

Step 1: Error Tracking (Sentry)

Prompt: "Install and configure Sentry for my React frontend and Express backend. Use environment variables for the DSN. Set up the React error boundary so unhandled errors get captured. Also configure source maps so I can see readable stack traces in production."

Step 2: Analytics (PostHog)

Prompt: "Add PostHog analytics to my React app. I want to automatically track page views, and I need custom events for: recipe_created, recipe_liked, comment_posted, and user_signed_up. Use environment variables for the PostHog API key."

Step 3: Email Notifications (Brevo)

Prompt: "Set up Brevo email sending in my Express backend. Create a service that sends a notification email when someone comments on a user's recipe. Use the Brevo SDK and store the API key in an environment variable. Include the commenter's name, the recipe title, and a link to view the comment."

Notice how each prompt is specific about what you want, mentions the tools you're using, and reminds the AI about environment variables. This is the pattern that gets you clean, production-ready integrations.

How Many Add-Ons Should You Actually Use?

It's easy to go overboard. Every add-on you install adds complexity, potential security surface area, and often cost as you scale. Here's a sensible progression:

MVP / Launch Phase:

- Cloudflare (CDN + security, free)
- Google Analytics (basic traffic insights, free)
- Sentry free tier (error tracking)
- One email service (Brevo or Resend)

Growing Phase (100+ users):

- Add product analytics (PostHog or Hotjar)
- Add live chat (Tawk.to or Crisp)
- Consider image optimization (Cloudinary)

Scaling Phase (1,000+ users):

- Full monitoring stack
- A/B testing tools
- Advanced email workflows
- Performance monitoring (Vercel Speed Insights)

Key Takeaways

- Build what's unique, integrate everything else. Your time is better spent on features that differentiate your app than on rebuilding solved problems.
- Start with the basics: Cloudflare, Google Analytics, Sentry, and an email service cover 80% of what most apps need at launch.
- Understand the four integration patterns (script tag, npm package, API, SDK) so you can evaluate any new tool quickly.
- Use AI to handle the integration work. Tools like Cursor and Claude Code are excellent at setting up third-party integrations because these tools have extensive documentation the AI has been trained on.
- Don't install everything at once. Add tools as you actually need them. Every integration adds complexity.

- Always use environment variables for API keys and configuration. Never hardcode secrets.
- Check the free tier first. Most developer tools offer generous free tiers that will last well beyond your MVP phase.
- Read the docs, or better yet, feed them to your AI. When integrating a new tool, paste the relevant documentation into your Cursor chat for the best results.

User Flow Analysis: Understanding How People Actually Use Your App

"Imagine you own a physical shop. You've arranged the shelves beautifully, put the bestsellers right at the front, and designed a lovely checkout counter. But then you install a security camera and realise something shocking: most customers walk in, turn right, wander around confused for 30 seconds, and walk straight back out. They never even see your bestsellers. User flow analysis is that security camera for your web app."

Building an app is one thing. Understanding how people actually use it is something else entirely. You might assume users follow the logical path you designed: land on the homepage, read about the features, sign up, complete onboarding, start using the core feature. But in reality, users are unpredictable. They skip steps, get confused, click things you never expected, and abandon the app at the strangest moments.

User flow analysis gives you the data to stop guessing and start knowing. And the good news? Modern tools make this accessible even if you've never looked at an analytics dashboard before.

What Is a User Flow (And Why Should You Care)?

A user flow is the path a person takes through your app from the moment they arrive to the moment they leave (or ideally, the moment they complete the action you designed the app for). Every app has a "happy path," which is the ideal journey you want users to take:

```
Landing Page --> Sign Up --> Onboarding --> Core Action --> Retention Loop
```

For example, if you built a recipe app:

```
Homepage --> Browse Recipes --> Sign Up --> Save First Recipe --> Get Weekly Email  
--> Come ...
```

Or if you built a SaaS tool:

```
Landing Page --> Free Trial Signup --> Complete Setup --> Use Core Feature -->  
Upgrade to Paid
```

The problem is that the actual flow usually looks more like this:

```
Landing Page --> Pricing Page --> Back to Landing --> Sign Up --> Get Confused at  
Onboarding...
```

User flow analysis helps you identify exactly where people get stuck, confused, or bored, and then fix those specific points. It's one of the highest-impact things you can do for your app because even small improvements at key drop-off points can dramatically increase the number of people who actually use your product.

The User Journey Map: Your Starting Framework

Before you plug in any tools, it helps to map out the ideal user journey you want people to follow. This gives you a baseline to measure against.

Ask your AI to help:

Prompt for Cursor: "Help me create a user journey map for my [describe app]. I need to map out the stages from first visit to becoming a regular user. For each stage, list the page/screen they'd see, the action they should take, and what might cause them to drop off."

A good user journey map covers these stages:

Stage	Goal	Key Metric
Awareness	User lands on your site	Unique visitors, traffic sources
Activation	User signs up or starts trial	Signup conversion rate
Onboarding	User completes initial setup	Onboarding completion rate
Core Action	User does the main thing your app offers	Feature adoption rate
Retention	User comes back and uses it again	Day 7 / Day 30 return rate
Revenue	User upgrades to paid	Free-to-paid conversion rate
Referral	User tells others about it	Invite rate, shares

This framework (sometimes called "pirate metrics" or AARRR) gives you a structured way to think about where users are falling off.

Tools for User Flow Analysis

There's a rich ecosystem of tools available, and many have generous free tiers. Here's how they compare:

Tool	Best For	Free Tier	Standout Feature	Integration
Hotjar	Visual insights (heatmaps + recordings)	Basic plan free	Session recordings you can watch	Script tag
PostHog	Product analytics + feature flags	1M events/mo	Open source, self-hostable	npm package
Google Analytics 4	Traffic analysis + conversion funnels	Unlimited (free)	Integration with Google ecosystem	Script tag
Mixpanel	Event-based analytics + funnels	20M events/mo	Powerful cohort analysis	npm / API
Amplitude	Product analytics + experimentation	50K tracked users/mo	Behavioral cohorting	npm / API
Microsoft Clarity	Heatmaps + session recordings	Completely free	AI-powered session insights	Script tag

For most non-coder builders, I'd recommend this combination:

- Google Analytics 4 for traffic and acquisition data (where users come from)
- PostHog or Hotjar for product analytics (what users do inside your app)
- Add Mixpanel or Amplitude later if you need deeper funnel analysis

Setting Up Event Tracking (The Foundation of Everything)

Event tracking is how you tell your analytics tool, "Hey, something important just happened." Every meaningful action in your app should be tracked as an event.

Common events to track:

Event Name	When to Fire	Why It Matters
page_view	User navigates to a new page	Basic engagement metric
signup_started	User clicks "Sign Up" button	Measures interest
signup_completed	User finishes registration	Conversion metric
onboarding_step_1	User completes first onboarding	Tracks onboarding progress
onboarding_completed	User finishes all onboarding	Activation metric
core_action_completed	User does the main thing (e.g., creates a project)	Value delivery metric
upgrade_clicked	User clicks upgrade/pricing	Purchase intent
subscription_started	User completes payment	Revenue metric

Prompt for Cursor: "Add PostHog event tracking to my React app. I need to track these events: signup_started, signup_completed, onboarding_step_1 through onboarding_step_3, onboarding_completed, first_project_created, and upgrade_clicked. Use the PostHog React SDK with the API key stored in an environment variable. Also set up automatic page view tracking."

Here's what PostHog event tracking looks like in a React component:

```
import posthog from 'posthog-js';

// Track when a user completes signup
const handleSignupComplete = (user) => {
  posthog.capture('signup_completed', {
    method: 'email', // or 'google', 'github'
    referral_source: user.referralSource,
  });
};

// Track onboarding progress
const handleOnboardingStep = (stepNumber) => {
  posthog.capture(`onboarding_step_${stepNumber}`, {
    time_since_signup: getTimeSinceSignup(),
  });
};
```

The key insight here is that you're not just tracking that something happened, but also attaching useful properties (like how the user signed up, or how long it took) that help you understand the context.

Heatmaps and Session Recordings: Watching Your Users (Ethically)

Heatmaps and session recordings are perhaps the most eye-opening tools available. They show you exactly what users are doing on your pages.

Heatmaps aggregate data from many users to show:

- Click maps: Where people are clicking (and where they're clicking but nothing happens)
- Scroll maps: How far down the page people actually scroll (spoiler: often not very far)
- Move maps: Where the cursor hovers (correlates roughly with attention)

Session recordings let you watch individual user sessions as if you were looking over their shoulder. You'll see:

- Where they hesitate
- Where they rage-click (clicking the same thing repeatedly because it's not working)
- Where they get confused and backtrack
- The exact moment they give up and leave

Hotjar and Microsoft Clarity both offer session recordings. Clarity is completely free with no session limits, making it an excellent starting point.

Prompt for Cursor: "Add Microsoft Clarity to my React app for session recordings and heatmaps. Here's my Clarity project ID: [YOUR_ID]. Make sure it only loads in production, not during development."

Conversion Funnels: Finding Where Users Drop Off

A conversion funnel visualizes how many users make it from one step to the next. The "funnel" metaphor is apt because you always lose people at each step. The goal is to make the funnel as wide as possible (lose fewer people).

Here's an example funnel for a SaaS app:

Step	Users	Drop-off
Visited landing page	1,000	-
Clicked "Sign Up"	120	88% left
Completed registration	80	33% left

Started onboarding	60	25% left
Completed onboarding	30	50% left
Used core feature	20	33% left
Returned next week	8	60% left

Looking at this data, two things jump out:

1. 88% of visitors don't even click Sign Up -- your landing page might need work
2. 50% drop off during onboarding -- your onboarding is too long or confusing

These are the kind of insights that transform your app. Without funnel data, you might spend weeks polishing a feature that 20 people use, while the real problem is that hundreds of people can't even figure out how to get started.

Google Analytics 4, PostHog, and Mixpanel all let you build custom funnels. Set them up for your most critical user journeys.

A/B Testing: Letting Data Make Decisions

Once you've identified a drop-off point, A/B testing helps you figure out what fix actually works. Instead of guessing whether a blue button or a green button converts better, you show version A to half your users and version B to the other half, then measure which performs better.

Common things to A/B test:

- Headline text on your landing page
- Button color, text, and placement
- Length of your signup form (fewer fields vs. more context)
- Onboarding flow (guided tour vs. self-exploration)
- Pricing page layout

PostHog has built-in A/B testing (they call it "Experimentation"), and it integrates directly with their analytics. This means you can set up a test and immediately see how it affects your conversion funnel.

Prompt for Cursor: "Help me set up an A/B test using PostHog's experimentation feature. I want to test two versions of my signup page headline. Version A says 'Start Building Today' and version B says 'Create Your First App in 10 Minutes'. Track which version leads to more completed signups."

Identifying and Fixing Drop-Off Points

Once you have data flowing, here's a practical process for using it:

Step 1: Build your main funnel (landing page through core action)

Step 2: Identify the biggest drop-off (where are you losing the most people?)

Step 3: Watch session recordings of users who dropped off at that point. Look for patterns:

- Are they confused by the interface?
- Is something not loading or broken?
- Are they distracted by something irrelevant?
- Is the page asking too much of them?

Step 4: Form a hypothesis ("Users drop off at onboarding step 2 because we ask for too much information")

Step 5: Make a change and measure (simplify the form, then check if the drop-off rate improves)

Step 6: Repeat (move on to the next biggest drop-off)

This cycle, sometimes called "growth experimentation," is how successful products continuously improve. And the best part? Your AI tools can help at every step.

Prompt for Claude Code or Cursor: "Analyze my PostHog funnel data. Users are dropping off 45% at the onboarding screen where we ask them to connect their GitHub account. Suggest 3 alternative approaches that might reduce this drop-off, and help me implement the most promising one."

Using Data to Drive Product Decisions

User flow analysis isn't just about fixing bugs or tweaking button colors. It fundamentally changes how you make product decisions:

- Feature prioritization: Build features that your most active users actually want, not what you think they want
 - Removing features: If analytics show nobody uses a feature, remove it. Simpler is better.
 - Pricing decisions: If users consistently drop off at the pricing page, your pricing might be too high or poorly communicated
 - Content strategy: If users who read your onboarding guide retain 3x better, invest in better documentation
 - Marketing focus: If users from Twitter convert 5x better than users from Facebook ads, redirect your marketing budget
-

Privacy and Ethics: Being a Good Data Citizen

Before you go wild with tracking, a few important notes:

- Always have a privacy policy that discloses what you track. Tools like Termly or Iubenda can generate one for you.
 - Respect cookie consent laws (GDPR in Europe, CCPA in California). PostHog and others have consent management built in.
 - Don't track personally identifiable information (PII) unless you absolutely need it. Track events and behaviors, not personal details.
 - Session recordings can capture sensitive data. Configure your tools to mask input fields, especially password and payment fields.
 - Give users the option to opt out. It's both ethical and increasingly a legal requirement.
-

Practical Setup: Your First Analytics Stack

Here's a step-by-step plan to get user flow analysis running in your app:

1. Add Google Analytics 4 for basic traffic and acquisition data
2. Add PostHog (or Hotjar) for product analytics and session recordings
3. Define your key events (signup, onboarding steps, core action, upgrade)
4. Build your first funnel (landing page through core action completion)
5. Watch 10 session recordings of users who dropped off at your biggest funnel leak
6. Make one improvement based on what you observed
7. Measure the impact over the next week

Prompt for Cursor: "Help me set up a complete analytics stack for my React app. I need Google Analytics 4 for traffic data and PostHog for product analytics. Set up automatic page view tracking, identify users after login, and create custom events for my key conversion points: signup, onboarding completion, and first project creation."

Key Takeaways

- User flow analysis is your app's security camera. It shows you what users actually do, not what you assume they do.
- Map your ideal user journey first, then use analytics to see where reality diverges from the plan.
- Start with event tracking. Every meaningful user action should be captured as an event with useful properties.
- Heatmaps and session recordings are the fastest way to understand why users are dropping off, not just where.
- Conversion funnels reveal your biggest opportunities. Fix the largest drop-off point first for maximum impact.

- A/B testing removes guesswork. When you're unsure which approach is better, test both and let the data decide.
- Follow the cycle: Identify drop-off, watch recordings, form hypothesis, make change, measure impact, repeat.
- Respect user privacy. Disclose your tracking, respect consent laws, mask sensitive inputs, and let users opt out.
- You don't need every tool. Google Analytics plus PostHog (or Hotjar) covers most needs. Add more tools as you grow.
- AI can help at every step, from setting up tracking code to analyzing the data and suggesting improvements.

Crawling Other Websites: Extracting Data From the Web

"Imagine you're a researcher who needs to check the price of a specific product at 50 different stores every single day. You could drive to each store, walk in, find the product, write down the price, and move on to the next one. Or... you could send a very efficient robot to do it for you in about 3 seconds. Web crawling is that robot."

At some point in your app-building journey, you'll probably want to pull data from another website. Maybe you want to monitor competitor prices, aggregate news articles, collect public job listings, or pull product reviews for analysis. This is called web crawling (or web scraping), and it's one of those superpowers that opens up entirely new categories of applications.

But with great power comes great responsibility. Web crawling sits in a fascinating gray area of technology, law, and ethics, so we need to cover the rules of the road before we start building scrapers.

What's the Difference Between Crawling and Scraping?

People use these terms interchangeably, but there's a subtle distinction:

- Web Crawling is about discovering pages. Think of Google's bots systematically following links across the entire internet to find new content. A crawler navigates from page to page.
- Web Scraping is about extracting data from specific pages. Once you've found the page you want, scraping pulls out the structured information (prices, titles, dates, etc.).

In practice, most projects involve both: you crawl to find the right pages, then scrape to extract the data you need.

The Legal and Ethical Landscape (Read This First)

Before you build anything, you need to understand the rules. Web scraping isn't inherently illegal, but it can cross legal lines depending on what you scrape, how you scrape it, and what you do with the data.

Things that are generally OK:

- Scraping publicly available data that anyone can see without logging in
- Respecting the website's robots.txt file (more on this below)
- Making requests at a reasonable rate (not hammering the server)
- Scraping for personal research or analysis

Things that can get you in trouble:

- Scraping data behind a login wall (especially if you agreed to Terms of Service that prohibit it)
- Ignoring robots.txt restrictions
- Scraping personal data (names, emails, phone numbers) without consent, which can violate GDPR and similar privacy laws
- Overloading a website's servers with too many requests (this is essentially a denial-of-service attack)
- Scraping copyrighted content and republishing it as your own
- Scraping in violation of a website's Terms of Service when you have an account

Understanding robots.txt

Most websites have a file at `yoursite.com/robots.txt` that tells automated bots what they're allowed to crawl. Here's a real example:

```
User-agent: *  
Allow: /  
Disallow: /admin/  
Disallow: /private/  
Disallow: /api/  
  
Sitemap: https://example.com/sitemap.xml
```

This says: "Any bot can crawl most of the site, but stay away from `/admin/`, `/private/`, and `/api/`." Always check and respect robots.txt before scraping any site.

Prompt for Cursor: "Before I start scraping, help me write a Node.js function that fetches and parses a website's robots.txt file to check whether a specific URL path is allowed for scraping."

The Golden Rules of Ethical Scraping

1. Check robots.txt first and respect what it says
2. Read the Terms of Service if you have an account on the site
3. Rate limit your requests (1-2 requests per second maximum, add delays between requests)
4. Identify yourself with a custom User-Agent string so the site owner can contact you if needed
5. Don't scrape personal data without a legitimate legal basis
6. Cache results so you don't scrape the same page repeatedly
7. If an API exists, use it instead. APIs are the "front door" while scraping is climbing through a window

The Tools of the Trade

There are several approaches to web scraping, each suited to different situations:

Tool	Best For	Type	Difficulty	Handles JavaScript?
Cheerio + Axios	Static HTML pages	Node.js library	Easy	No
Puppeteer	JavaScript-heavy sites (SPAs)	Headless Chrome	Medium	Yes
Playwright	Cross-browser	Headless browser	Medium	Yes
Apify	Managed scraping at scale	Cloud service	Easy (but costs \$)	Yes
Scrapy	Large-scale crawling projects	Python framework	Medium-Hard	No (without plugins)
Crawlee	Node.js crawling framework	Library (by Apify)	Medium	Yes

When to Use What

Use Cheerio + Axios when:

- The page content is in the HTML source (right-click > View Source shows the data)
- You don't need to click buttons, scroll, or interact with the page
- You want the simplest, fastest solution

Use Puppeteer or Playwright when:

- The content loads dynamically via JavaScript (many modern sites)
- You need to interact with the page (click buttons, fill forms, scroll)
- The site uses client-side rendering (React, Vue, Angular apps)

Use Apify when:

- You need to scrape at scale (thousands of pages)
- You want someone else to handle proxies, rate limiting, and infrastructure
- You don't want to manage servers running headless browsers

Building Your First Scraper: Cheerio + Axios

Let's start with the simplest approach. Cheerio is a library that lets you parse HTML using a syntax similar to jQuery (a popular JavaScript library). Axios handles making the HTTP requests.

Prompt for Cursor: "Build a Node.js script using Cheerio and Axios that scrapes product names and prices from a public e-commerce page. Save the results to a JSON file. Include error handling, respect rate limits with a delay between requests, and add a custom User-Agent header."

Here's what the resulting code might look like:

```

import axios from 'axios';
import * as cheerio from 'cheerio';
import fs from 'fs';

interface Product {
  name: string;
  price: string;
  url: string;
}

async function scrapeProducts(url: string): Promise<Product[]> {
  const response = await axios.get(url, {
    headers: {
      'User-Agent': 'MyAppBot/1.0 (contact@myapp.com)',
    },
  });

  const $ = cheerio.load(response.data);
  const products: Product[] = [];

  $('.product-card').each((index, element) => {
    products.push({
      name: $(element).find('.product-title').text().trim(),
      price: $(element).find('.product-price').text().trim(),
      url: $(element).find('a').attr('href') || '',
    });
  });

  return products;
}

// Save results
const products = await scrapeProducts('https://example-shop.com/products');
fs.writeFileSync('products.json', JSON.stringify(products, null, 2));
console.log(`Scraped ${products.length} products`);

```

The key concepts here:

- `axios.get()` fetches the page HTML, just like a browser would
- `cheerio.load()` parses that HTML so you can navigate it
- CSS selectors (`.product-card`, `.product-title`) target specific elements, just like CSS styling
- `.text().trim()` extracts the text content and removes whitespace

Handling Dynamic Content with Puppeteer

Many modern websites use JavaScript to render content. If you "View Source" on the page and the data isn't there, but it appears when the page loads in a browser, you need a headless browser.

Puppeteer launches a real Chrome browser (without the visible window) and controls it programmatically:

```
import puppeteer from 'puppeteer';

async function scrapeDynamicPage(url: string) {
  const browser = await puppeteer.launch({ headless: true });
  const page = await browser.newPage();

  // Set a custom user agent
  await page.setUserAgent('MyAppBot/1.0 (contact@myapp.com)');

  // Navigate and wait for content to load
  await page.goto(url, { waitUntil: 'networkidle2' });

  // Wait for the specific element to appear
  await page.waitForSelector('.product-list');

  // Extract data from the rendered page
  const products = await page.evaluate(() => {
    const items = document.querySelectorAll('.product-card');
    return Array.from(items).map(item => ({
      name: item.querySelector('.title')?.textContent?.trim() || '',
      price: item.querySelector('.price')?.textContent?.trim() || '',
    }));
  });

  await browser.close();
  return products;
}
```

Puppeteer is more powerful but also more resource-intensive. Each scrape launches an actual browser instance, which uses significantly more memory and CPU than a simple HTTP request.

Prompt for Cursor: "Help me build a Puppeteer scraper that handles pagination. It should scrape all products from an e-commerce site by clicking the 'Next Page' button until there are no more pages, collecting data from each page. Add a 2-second delay between pages to be respectful to the server."

Handling Pagination

Most real-world scraping involves multiple pages. There are two common patterns:

URL-based pagination (easier):

```
https://example.com/products?page=1
https://example.com/products?page=2
https://example.com/products?page=3
```

You just loop through the URLs:

```
const allProducts = [];
for (let page = 1; page <= totalPages; page++) {
  const products = await scrapeProducts(`https://example.com/products?page=${page}`);
  allProducts.push(...products);
  // Be respectful: wait 2 seconds between requests
  await new Promise(resolve => setTimeout(resolve, 2000));
}
```

"Load More" button pagination (needs headless browser):

Some sites use infinite scroll or "Load More" buttons. For these, you need Puppeteer or Playwright to click the button and wait for new content to appear.

Storing Scraped Data

Once you've scraped data, you need to store it somewhere useful:

Storage Option	Best For	Complexity
JSON file	Small, one-time scrapes	Very simple
CSV file	Data you'll open in Excel/Sheets	Simple
SQLite database	Medium datasets, local analysis	Medium
Supabase/PostgreSQL	Data your app will use	Medium
Google Sheets API	Collaborative review of scraped data	Medium

For most app-building scenarios, storing directly in your Supabase database makes the most sense because your app can then display and use the data immediately.

Prompt for Cursor: "Modify my scraper to store results directly in my Supabase database instead of a JSON file. Create a 'scraped_products' table with columns for name, price, source_url, and scraped_at timestamp. Use upsert so re-running the scraper updates existing products instead of creating duplicates."

Scheduling Scrapers (Cron Jobs)

If you need to scrape regularly (daily price updates, hourly news aggregation), you'll want to schedule your scraper to run automatically.

Option 1: Simple cron job (for your own server)

A cron job is like setting an alarm clock for your code. On Railway or any Node.js server, you can use libraries like node-cron:

```
import cron from 'node-cron';

// Run every day at 6:00 AM
cron.schedule('0 6 * * *', async () => {
  console.log('Starting daily scrape...');
  await runScraper();
  console.log('Scrape complete!');
});
```

Option 2: GitHub Actions (free for public repos)

You can set up a GitHub Action that runs your scraper on a schedule and commits the results back to your repository.

Option 3: Managed services

Apify and similar platforms handle scheduling, proxies, and scaling for you.

Real-World Use Cases

Here are practical scenarios where non-coders building apps might use web scraping:

Price Monitoring:

Build an app that tracks product prices across multiple retailers and alerts users when something drops below their target price.

Content Aggregation:

Create a niche news aggregator that pulls headlines and summaries from multiple sources in your industry.

Competitor Analysis:

Monitor competitor websites for new features, pricing changes, or job postings (which can indicate what they're building).

Lead Generation:

Scrape business directories (that allow it) to build lists of potential customers for a B2B product.

Research Data Collection:

Gather publicly available data for analysis, like job posting trends, real estate listings, or restaurant menus.

When to Use an API Instead

Before building a scraper, always check if the website offers an API. APIs are:

- More reliable (they don't break when the site redesigns)
- Faster (you get structured data directly)
- Legal (the site explicitly allows this access)
- Easier to maintain (no CSS selectors to update)

Many services you might think of scraping actually have APIs:

- Twitter/X has an API for tweets
- Reddit has an API for posts and comments
- Amazon has a Product Advertising API
- Google has APIs for Maps, Search, YouTube, and more
- Yelp, Glassdoor, LinkedIn all have official APIs (with varying access levels)

Prompt for Cursor: "I want to get restaurant reviews for my app. Before I build a scraper, help me research whether the major review platforms (Google, Yelp, TripAdvisor) offer public APIs. Compare what data is available through their APIs versus what I'd need to scrape."

Common Pitfalls and How to Avoid Them

Problem	Solution
Site blocks your requests	Add delays, rotate User-Agents, use proxies
HTML structure changes	Use resilient selectors, add error handling, monitor for failures
JavaScript-rendered content not appearing	Switch from Cheerio to Puppeteer/Playwright
Scraper gets CAPTCHAs	Use a managed service like Apify that handles CAPTCHAs
Data is inconsistent or messy	Add data cleaning/validation step after scraping
Running out of memory (Puppeteer)	Close browser instances properly, limit concurrent pages
Getting IP banned	Implement respectful rate limiting, use proxy rotation

Building a Scraper with AI: A Complete Workflow

Here's how to approach a scraping project using your AI coding tools:

1. Research first: Check for APIs, read the site's Terms of Service and robots.txt
2. Inspect the page: Right-click the data you want, choose "Inspect Element" and take a screenshot
3. Share with your AI: Give Cursor or Claude Code a screenshot of the page and the HTML structure

4. Start simple: Build a scraper for one page first, get that working perfectly
5. Add pagination: Extend to handle multiple pages
6. Add storage: Save results to your database or file system
7. Add scheduling: If needed, set up automated runs
8. Add monitoring: Log results and alert yourself if the scraper fails

Prompt for Cursor: "I want to build a scraper for [website URL]. Here's a screenshot of the data I need. First, check if they have a public API. If not, inspect the page structure and build a Cheerio + Axios scraper that extracts [specific data fields]. Save results to a JSON file and include proper error handling, rate limiting (2 seconds between requests), and a custom User-Agent."

Key Takeaways

- Web scraping extracts data from websites automatically, saving you from manual copy-paste work across hundreds of pages.
- Always check for an API first. APIs are more reliable, legal, and easier to maintain than scrapers.
- Respect the rules: Check robots.txt, read Terms of Service, rate limit your requests, and never scrape personal data without consent.
- Start with Cheerio + Axios for simple static pages. Move to Puppeteer or Playwright when you need to handle JavaScript-rendered content.
- Handle pagination from the start, as real-world data almost always spans multiple pages.
- Store data wisely: JSON files for quick experiments, your Supabase database for data your app will use.
- Schedule scrapers with cron jobs or managed services for regular data collection.
- Managed services like Apify handle the hard parts (proxies, CAPTCHAs, scaling) if you need to scrape at scale.
- AI tools are excellent at writing scrapers. Share a screenshot of the page and describe what data you need, and Cursor or Claude Code will generate the extraction code.
- Monitor your scrapers. Websites change their HTML structure regularly, so your scraper will need occasional maintenance.

Code Validation: Making Sure Your Code Actually Works Well

"Imagine you've just finished writing a 20-page essay with the help of a very talented ghostwriter. The ideas are brilliant, the structure makes sense, and the conclusions are compelling. But what if there are typos on every page, inconsistent formatting, and one paragraph accidentally contradicts another? You'd want someone to proofread it before publishing, right? Code validation is that proofreader for your AI-generated code."

Here's something that might surprise you: even the best AI coding tools produce code that has issues. Not always bugs that crash your app, but subtle problems. Inconsistent formatting, unused variables, functions that could break with unexpected input, missing error handling, or patterns that will cause headaches as your project grows. This isn't a criticism of AI. It's just reality. AI generates code one piece at a time, and it doesn't always maintain perfect consistency across an entire project.

Code validation is a set of tools and practices that automatically catch these issues before they become real problems. Think of it as a safety net that catches mistakes, whether they're made by you, your AI assistant, or that version of you who was coding at midnight after too much coffee.

Why Code Quality Matters (Even for AI-Generated Code)

You might wonder: "If my app works, why does code quality matter?" Great question. Here's why:

1. Bugs hide in messy code. When code is inconsistent and poorly structured, bugs find more places to hide. Clean code is easier to debug.
2. AI works better with clean code. When you ask Cursor or Claude Code to modify clean, well-structured code, the results are dramatically better than when the codebase is a mess.
3. Future-you will thank present-you. Coming back to a project after a month? Clean, validated code with consistent patterns is infinitely easier to understand.
4. Collaboration becomes possible. If you ever bring on a co-founder, freelancer, or contributor, validated code means they can get productive faster.
5. Deployment platforms are pickier. Many hosting services run checks during deployment. Code that passes validation locally will deploy more smoothly.

The Code Quality Toolchain: Your Four Guardians

There are four main tools that work together to keep your code clean. Think of them as layers of defense:

Tool	What It Does	Catches	Analogy
ESLint	Finds problematic patterns and potential bugs	Unused variables, missing dependencies, passing a string where a number is expected, missing properties	Grammar checker
TypeScript	Ensures data types are correct		Spell checker on steroids

Prettier	Formats code consistently	Inconsistent indentation, spacing, line lengths	Auto-formatter
Husky + lint-staged	Runs checks before every commit	Prevents bad code from entering your repository	Bouncer at the door

Let's dive into each one.

ESLint: Your Automated Code Reviewer

ESLint scans your code and flags potential problems. It's like having a senior developer looking over your shoulder, but one who never gets tired and checks every single file.

Common things ESLint catches:

- Unused variables: You declared `const userName = 'Dave'` but never used it anywhere
- Missing dependencies in React hooks: Your `useEffect` uses a variable but doesn't list it in the dependency array (a very common source of bugs)
- Inconsistent comparisons: Using `==` instead of `===` (which can cause subtle type coercion bugs in JavaScript)
- Unreachable code: Code that exists after a `return` statement and will never execute
- Missing key props: React list items without unique key props (causes rendering issues)

Prompt for Cursor: "Set up ESLint in my React + Vite + TypeScript project with the recommended rules for React and TypeScript. Include the React Hooks plugin to catch missing dependencies in useEffect. Also configure it to work nicely with Prettier so they don't conflict."

Most Vite projects already come with ESLint pre-configured, but it's worth checking that the rules are sensible. Here's what a typical ESLint config looks like:

```
// eslint.config.js (flat config format)
import js from '@eslint/js';
import tseslint from 'typescript-eslint';
import react from 'eslint-plugin-react';
import reactHooks from 'eslint-plugin-react-hooks';

export default [
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    plugins: {
      react,
      'react-hooks': reactHooks,
    },
    rules: {
      'react-hooks/rules-of-hooks': 'error',
      'react-hooks/exhaustive-deps': 'warn',
      'no-unused-vars': 'warn',
      'no-console': 'warn',
    },
  },
];
```

Running ESLint is simple:

```
# Check all files
npx eslint src/

# Auto-fix what can be fixed automatically
npx eslint src/ --fix
```

The --fix flag is magical. Many ESLint issues can be fixed automatically, which means you can clean up a messy codebase in seconds.

TypeScript: Your Type Safety Net

If you're building with the stack we teach in this course (React + Vite + TypeScript), you already have TypeScript. But you might not be getting its full benefits.

TypeScript's job is to make sure data flows through your app correctly. If a function expects a number, TypeScript screams at you when you accidentally pass it a string. If an object should have a name property, TypeScript won't let you access naem (with a typo).

Strict mode is where TypeScript really shines. By default, TypeScript is relatively lenient. Enabling strict mode catches significantly more issues:

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

What strict mode catches that normal TypeScript doesn't:

Check	What It Prevents
strictNullChecks	Accessing properties on values that might be null or undefined
noImplicitAny	Variables without explicit types (forces you to be clear about data shapes)
strictFunctionTypes	Passing the wrong type of callback function
strictPropertyInitialization	Class properties that aren't initialized

Prompt for Cursor: "Enable TypeScript strict mode in my project and fix all the type errors that appear. For each error, explain what was wrong and why the fix is important."

When you enable strict mode on an existing project, you'll likely get a wave of errors. Don't panic! Most are easy fixes, and your AI tools can handle them quickly. The result is a much more robust codebase.

Prettier: Consistent Formatting Without Arguments

Prettier is an "opinionated code formatter." Instead of debating whether to use tabs or spaces, single quotes or double quotes, trailing commas or not, Prettier makes the decision for you and formats all your code consistently.

This might sound trivial, but consistent formatting:

- Makes code easier to read and review
- Eliminates meaningless formatting changes in your Git history
- Ensures AI tools generate code that matches your existing style
- Stops you from wasting mental energy on formatting decisions

```
// .prettierrc
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2
}
```

Once configured, you can format your entire codebase in one command:

```
npx prettier --write src/
```

Prompt for Cursor: "Add Prettier to my project with sensible defaults. Configure it to work alongside ESLint without conflicts (install eslint-config-prettier). Set up a format script in package.json that formats all TypeScript and TSX files."

Husky + lint-staged: The Bouncer at the Door

Here's where it all comes together. Husky lets you run scripts automatically at certain Git events (like before a commit). lint-staged runs your linters only on the files you've changed (so it's fast).

The combination means: every time you try to commit code, ESLint checks for problems and Prettier formats your code. If there are errors, the commit is blocked until you fix them.

This is incredibly powerful because it means bad code literally cannot enter your repository.

Prompt for Cursor: "Set up Husky and lint-staged in my project. Configure a pre-commit hook that runs ESLint and Prettier on all staged TypeScript and TSX files. If ESLint finds errors, the commit should be blocked."

Here's what the configuration looks like:

```
// package.json (partial)
{
  "lint-staged": {
    "*.ts,tsx": [
      "eslint --fix",
      "prettier --write"
    ],
    "*.json,md,css": [
      "prettier --write"
    ]
  }
}
```

Now when you run git commit, the pre-commit hook automatically:

- 1. Takes your staged files
- 2. Runs ESLint with auto-fix on TypeScript files
- 3. Runs Prettier on all matching files
- 4. If ESLint found unfixable errors, blocks the commit
- 5. If everything passes, the commit goes through

Testing: Does Your Code Do What You Think It Does?

Linting and formatting catch style issues and potential bugs, but testing verifies that your code actually produces the correct results. There are different levels of testing:

Type	What It Tests	Tools	Effort
Unit Tests	Individual functions in isolation	Vitest, Jest	Low
Component Tests	React components render correctly	React Testing Library	Medium
Integration Tests	Multiple pieces working together	Vitest, Supertest	Medium
End-to-End Tests	Full user workflows in a browser	Playwright, Cypress	High

For non-coders building with AI, start with unit tests for your utility functions and component tests for your most important React components. You don't need 100% test coverage. Focus on the parts that would hurt the most if they broke.

Writing Your First Unit Test

Vitest is the best testing tool for Vite projects because it shares the same configuration:

```
// src/utils/formatPrice.ts
export function formatPrice(cents: number): string {
  return `$$${(cents / 100).toFixed(2)}`;
}

// src/utils/formatPrice.test.ts
import { describe, it, expect } from 'vitest';
import { formatPrice } from './formatPrice';

describe('formatPrice', () => {
  it('formats cents to dollar string', () => {
    expect(formatPrice(1999)).toBe('$19.99');
  });

  it('handles zero', () => {
    expect(formatPrice(0)).toBe('$0.00');
  });

  it('handles single digit cents', () => {
    expect(formatPrice(5)).toBe('$0.05');
  });
});
```

The pattern is simple: describe groups related tests, it defines a single test case, and expect checks that the result matches what you expected.

Prompt for Cursor: "Set up Vitest in my React + Vite project. Write unit tests for all the utility functions in my src/lib/ and src/utils/ directories. For each function, test the happy path, edge cases, and error cases."

Testing React Components

React Testing Library lets you test components the way users interact with them:

```
import { render, screen, fireEvent } from '@testing-library/react';
import { LoginButton } from './LoginButton';

it('shows login form when button is clicked', () => {
  render(<LoginButton />);
  fireEvent.click(screen.getByText('Log In'));
  expect(screen.getByLabelText('Email')).toBeInTheDocument();
});
```

Notice how the test reads almost like English: "Render the LoginButton, click the thing that says 'Log In', and check that an Email field appeared." This approach tests behavior rather than implementation, which means your tests won't break every time you refactor the component's

internals.

Reviewing AI-Generated Code (Even Solo Developers Should Do This)

One practice that dramatically improves code quality is actually reviewing what your AI generates before accepting it. It's tempting to just click "Accept All" and move on, but spending 30 seconds scanning the changes catches a surprising number of issues.

What to look for when reviewing AI output:

- Does it handle errors? What happens if the API call fails? If the user input is empty?
- Are there hardcoded values that should be environment variables or constants?
- Does it match existing patterns in your codebase? (naming conventions, file structure)
- Is it doing more than you asked? AI sometimes over-engineers or adds features you didn't request
- Are there security issues? Exposed API keys, SQL injection risks, missing input validation

Prompt for Cursor: "Review the code you just generated and check for: error handling gaps, hardcoded values that should be environment variables, security issues, and any deviations from the patterns used in the rest of this project."

Asking the AI to review its own output is a surprisingly effective technique. It often catches issues it missed on the first pass.

CI/CD: Automated Checks on Every Push

The final level of code validation is Continuous Integration (CI), which runs your checks automatically every time you push code to GitHub. GitHub Actions makes this free for public repositories and generous for private ones.

A basic CI pipeline runs:

1. Install dependencies
2. Run ESLint
3. Run TypeScript type checking
4. Run your test suite
5. Build the project (catches build errors)

Prompt for Cursor: "Create a GitHub Actions workflow file that runs on every push and pull request. It should install dependencies, run ESLint, run TypeScript type checking with tsc --noEmit, run Vitest, and build the project. If any step fails, the workflow should fail."

Here's what a basic CI workflow looks like:

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm test
      - run: npm run build
```

With this in place, you get a green checkmark or red X on every push, giving you confidence that your code is solid before deploying.

Putting It All Together: The Complete Setup

Here's the order to set everything up in a new (or existing) project:

1. ESLint (catch potential bugs and bad patterns)
2. Prettier (consistent formatting)
3. eslint-config-prettier (make them play nice together)
4. Husky + lint-staged (block bad commits)
5. Vitest (add tests for critical functions and components)
6. GitHub Actions CI (automate checks on every push)

Prompt for Cursor: "Set up a complete code quality toolchain in my React + Vite + TypeScript project. I need ESLint with React and TypeScript plugins, Prettier configured to work alongside ESLint, Husky pre-commit hooks with lint-staged, and Vitest for testing. Also create a GitHub Actions CI workflow. Give me the step-by-step commands to install everything and the configuration files I need."

Key Takeaways

- AI-generated code needs validation just like human-written code. AI is brilliant but not perfect, and it doesn't always maintain consistency across a project.
- ESLint catches bugs and bad patterns before they cause problems. Run it with `--fix` to auto-resolve many issues.
- TypeScript strict mode is your best friend. It catches entire categories of bugs at development time instead of at runtime in front of users.
- Prettier eliminates formatting debates. Configure it once, run it everywhere, never think about tabs vs. spaces again.
- Husky + lint-staged prevents bad code from entering your repository. If it can't pass linting, it can't be committed.
- Start testing with unit tests for your utility functions. They're easy to write and catch real bugs.
- Always review AI-generated code before accepting. Spend 30 seconds scanning for error handling gaps, hardcoded values, and security issues.
- Ask the AI to review its own output. It's surprisingly effective at catching issues on a second pass.
- GitHub Actions CI runs your entire validation suite automatically on every push, catching issues before deployment.
- You don't need perfection. Even setting up just ESLint and Prettier will significantly improve your code quality. Add more tools as your project grows.

Performance Analysis: Making Your App Fast and Keeping Users Happy

"Picture this: you walk into a restaurant, sit down, and wait. One minute goes by. Five minutes. Ten minutes. Nobody has even brought you a menu. What do you do? You leave. That's exactly what happens when your web app is slow. Users don't send you a polite email explaining that your loading times are suboptimal. They just close the tab and go somewhere else. Forever."

Performance might not sound as exciting as adding new features, but it's one of the most impactful things you can work on. Google's own research shows that when a page load time goes from 1 second to 3 seconds, the probability of a user bouncing increases by 32%. At 5 seconds, it jumps to 90%. And it's not just user patience. Google literally uses page speed as a ranking factor for search results. Slow site? Lower in Google. Lower in Google? Fewer users. Fewer users? Less revenue. The math is brutally simple.

The good news? Performance analysis tools are free, the fixes are often straightforward, and your AI coding tools are excellent at implementing optimizations once you know what to fix.

Core Web Vitals: The Numbers Google Cares About

Google has defined three specific metrics called Core Web Vitals that measure real-world user experience. These aren't arbitrary. They're based on research into what actually frustrates users.

Metric	Full Name	What It Measures	Good	Needs Work	Poor
LCP	Largest Contentful Paint	How quickly the main content loads	< 2.5s	2.5s - 4s	> 4s
INP	Interaction to Next Paint	How quickly the page responds to user input	< 200ms	200ms - 500ms	> 500ms
CLS	Cumulative Layout Shift	How much the page layout shifts unexpectedly	< 0.1	0.1 - 0.25	> 0.25

Let's break these down in plain English

LCP (Largest Contentful Paint): This measures how long it takes for the biggest visible element on the page to appear. Usually this is a hero image, a heading, or a large block of text. If your user is staring at a blank white screen for 4 seconds, your LCP is terrible.

INP (Interaction to Next Paint): This replaced FID (First Input Delay) in 2024. It measures how quickly your app responds when a user clicks a button, types in a field, or interacts with anything. If there's a noticeable delay between clicking and something happening, INP is the metric that captures it.

CLS (Cumulative Layout Shift): Have you ever tried to click a button on a webpage, but just as you click, an ad loads above it and everything shifts down, and you accidentally click something else? That's layout shift, and it's infuriating. CLS measures how much your layout moves around unexpectedly during loading.

Lighthouse: Your Free Performance Report Card

The easiest way to start analyzing performance is with Lighthouse, which is built right into Google Chrome. No installation needed.

How to run a Lighthouse audit:

1. Open your app in Chrome
2. Right-click anywhere and select "Inspect" (or press F12)
3. Click the "Lighthouse" tab at the top of DevTools
4. Select "Performance" (and optionally Accessibility, SEO, Best Practices)
5. Click "Analyze page load"
6. Wait about 30 seconds for the report

You'll get a score from 0-100 for each category, plus specific recommendations for improvement. A score above 90 is considered good. Below 50 needs serious attention.

Important: Always test in an Incognito window with browser extensions disabled. Extensions can dramatically affect performance scores and give you misleading results.

Prompt for Cursor: "I ran a Lighthouse audit on my React app and got a performance score of 62. The main issues are: LCP of 4.2 seconds, large JavaScript bundles (1.8MB total), render-blocking resources, and unoptimized images. Help me fix each issue systematically, starting with the highest-impact improvements."

Lighthouse is great for a quick overview, but it tests under simulated conditions. For real-world performance data, you'll want to pair it with field data from tools like Google Search Console or Vercel Analytics.

Chrome DevTools Performance Tab: The Deep Dive

For more detailed analysis, the Performance tab in Chrome DevTools lets you record and analyze exactly what happens during page load, millisecond by millisecond.

How to use it:

1. Open DevTools (F12)
2. Go to the "Performance" tab
3. Click the record button (circle icon)
4. Reload the page or perform the action you want to analyze
5. Stop the recording
6. Analyze the waterfall chart

The waterfall shows you exactly when each resource loads, which scripts take the longest to execute, and where the browser is spending its time. Look for:

- Long tasks (orange/red blocks): JavaScript that blocks the main thread for more than 50ms
- Layout shifts: Visible in the experience section
- Network waterfall: Shows which resources load in what order

This level of detail is usually more than you need as a non-coder, but it's invaluable when you need to diagnose a specific performance issue. And you can always screenshot the results and share them with your AI.

Prompt for Cursor: "Here's a screenshot of my Chrome DevTools Performance recording. There's a long task that takes 800ms during page load. Help me identify what's causing it and suggest how to fix it."

Bundle Size: The Silent Performance Killer

One of the most common performance problems in React apps is bundle size. Every npm package you install, every component you create, and every image you import gets bundled into JavaScript files that the user's browser has to download and parse before your app works.

A good target for your main bundle is under 200KB gzipped. Many React apps ship 1MB+ without anyone realizing it.

How to analyze your bundle:

The rollup-plugin-visualizer (for Vite projects) generates a visual treemap of your bundle, showing exactly which packages take up the most space:

```
npm install -D rollup-plugin-visualizer
```

```
// vite.config.ts
import { visualizer } from 'rollup-plugin-visualizer';

export default defineConfig({
  plugins: [
    react(),
    visualizer({
      open: true,
      gzipSize: true,
      filename: 'bundle-analysis.html',
    }),
  ],
});
```

Run `npm run build` and a visual report opens in your browser, showing every dependency as a proportionally-sized rectangle. You'll often be shocked to discover that a library you barely use is taking up 30% of your bundle.

Prompt for Cursor: "Add the rollup-plugin-visualizer to my Vite config so I can analyze my bundle size. After running the build, analyze the results and suggest which packages are the biggest and how I can reduce the bundle size. Consider alternatives, tree-shaking, and lazy loading."

The Top Performance Killers (And How to Fix Them)

Here are the most common performance problems and their solutions:

1. Unoptimized Images

Images are often the largest assets on a page. A single unoptimized photo can be 5MB, which is larger than your entire JavaScript bundle.

Fixes:

- Use WebP format instead of PNG/JPEG (30-50% smaller files)
- Resize images to the actual display size (don't serve a 4000px image in a 400px container)
- Use responsive images with `srcset` so mobile users get smaller files
- Lazy load images below the fold (images the user can't see yet)

```
<!-- Lazy loading with responsive sizing -->
```

Prompt for Cursor: "Audit all images in my React app. For each image, check if it's optimized (WebP format, appropriate dimensions, lazy loaded if below the fold). Create a script that converts all PNG/JPEG images to WebP and generates multiple sizes for responsive loading."

2. Too Many (or Too Large) Dependencies

Every npm package you add increases your bundle size. Some common culprits:

Package	Bundle Impact	Lighter Alternative
moment.js	~300KB	date-fns (30KB with tree-shaking) or dayjs (2KB)
lodash (full)	~70KB	lodash-es (tree-shakeable) or native JS methods
Full icon libraries	~500KB+	Import individual icons only
chart.js	~200KB	lightweight-charts or SVG-based charts

Prompt for Cursor: "Analyze my package.json and identify packages that are large or have lighter alternatives. Check if I'm importing full libraries when I could be importing individual modules. Suggest specific replacements that would reduce my bundle size."

3. Render-Blocking Scripts and Styles

Scripts in your <head> that don't use async or defer block the page from rendering until they've downloaded and executed.

Fixes:

- Add async or defer to third-party scripts
- Load non-critical CSS asynchronously
- Inline critical CSS for above-the-fold content

4. Unnecessary React Re-renders

React re-renders components whenever their state or props change. Unnecessary re-renders waste CPU time and make the UI feel sluggish.

Common causes:

- Creating new objects/arrays in render (triggers re-render of children)
- Not memoizing expensive calculations
- Passing inline functions as props

React optimization tools:

```
// React.memo: Prevents re-render if props haven't changed
const ExpensiveList = React.memo(({ items }) => {
  return items.map(item => <ListItem key={item.id} item={item} />);
});

// useMemo: Caches expensive calculations
const sortedItems = useMemo(() => {
  return items.sort((a, b) => a.name.localeCompare(b.name));
}, [items]);

// useCallback: Caches function references
const handleClick = useCallback((id) => {
  setSelectedId(id);
}, []);
```

Prompt for Cursor: "Analyze my React components for unnecessary re-renders. Check for: inline object/array creation in render, missing React.memo on expensive list components, functions that should use useCallback, and calculations that should use useMemo. Show me which components re-render most frequently and how to fix them."

Lazy Loading vs Preloading: Know the Difference

Before we get into the code, let's make sure we understand the concept, because this is one of those things that sounds technical but is actually really intuitive.

Imagine you've just moved into a new house and you have 50 boxes stacked in the hallway. It's late, you're tired, and you just need your toothbrush. Do you:

- A) Open every single box right now — unpack your kitchen gadgets, your winter clothes, your book collection, your childhood photos — just to find that toothbrush?
- B) Walk over to the box labelled "Toiletries", open just that one, grab your toothbrush, and deal with the rest tomorrow?

Option A is preloading (also called eager loading). Your app loads everything upfront — every page, every component, every image — before the user even needs it. The result? A slow initial load that makes users stare at a blank screen.

Option B is lazy loading. Your app only loads what's needed right now and fetches the rest on demand. The result? A fast initial load and a snappy experience.

Aspect	Preloading (Eager)	Lazy Loading
Initial load time	Slow — everything downloads upfront	Fast — only essentials load first
When content loads	All at once, before user needs it	On demand, when user navigates to it
Best for	Tiny apps with minimal content	Most real-world apps with multiple pages
User experience	Longer wait upfront, instant after	Quick start, brief loads on navigation
Analogy	Opening all 50 boxes at once	Opening boxes as you need them

When is preloading actually useful? In small doses. You might preload the next page a user is likely to visit (like preloading page 2 of search results while they read page 1). Some frameworks call this prefetching. But loading your entire app upfront? Almost never a good idea.

Now let's see how to implement lazy loading in React...

Code Splitting and Lazy Loading Routes

Instead of loading your entire app upfront, code splitting breaks it into smaller chunks that load on demand. The most impactful place to split is at the route level.


```
import { lazy, Suspense } from 'react';

// Instead of:
import DashboardPage from './pages/DashboardPage';
import SettingsPage from './pages/SettingsPage';
import AdminPage from './pages/AdminPage';

// Use lazy loading:
const DashboardPage = lazy(() => import('./pages/DashboardPage'));
const SettingsPage = lazy(() => import('./pages/SettingsPage'));
const AdminPage = lazy(() => import('./pages/AdminPage'));

// Wrap in Suspense with a fallback
function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
        <Route path="/dashboard" element={<DashboardPage />} />
        <Route path="/settings" element={<SettingsPage />} />
        <Route path="/admin" element={<AdminPage />} />
      </Routes>
    </Suspense>
  );
}
```

With this approach, the admin page code is only downloaded when a user actually navigates to / admin. Users who never visit admin never download that code. This can dramatically reduce your initial bundle size.

Prompt for Cursor: "Convert all my page-level route components to use React.lazy() with dynamic imports. Add a Suspense boundary with a loading spinner component. Make sure the loading spinner matches my app's design."

CDN and Caching: Serving Content from Nearby Servers

A CDN (Content Delivery Network) stores copies of your static files (JavaScript, CSS, images) on servers around the world. When a user in Tokyo visits your app, they get files from a server in Tokyo instead of your server in Virginia. The result: dramatically faster load times.

If you're using Vercel, you already have a CDN. Vercel automatically serves your static assets from their global edge network. Cloudflare adds another layer of caching and protection if you configure it as your DNS provider.

Caching tells browsers to store files locally so they don't need to be downloaded again on repeat visits. Vite handles this well by default. It adds content hashes to filenames (like main.a1b2c3d4.js), which means:

- The browser caches the file indefinitely
 - When you deploy new code, the filename changes, so users get the new version
 - Unchanged files keep the same name and don't need to be re-downloaded
-

Database Query Optimization

If your app has a backend, slow database queries can bottleneck your entire app regardless of how optimized your frontend is.

Common database performance issues:

- Querying all columns when you only need a few (SELECT * vs SELECT id, name)
- Missing indexes on frequently queried columns
- N+1 queries (loading a list, then making a separate query for each item)
- Not paginating large result sets

Prompt for Cursor: "Analyze the Supabase queries in my Express backend. Check for: queries that select all columns when they don't need to, missing pagination on list endpoints, potential N+1 query patterns, and suggest database indexes that would improve performance based on my query patterns."

A Practical Performance Improvement Workflow

Here's a step-by-step process you can follow:

Step 1: Measure (don't guess)

- Run Lighthouse in an Incognito window
- Note your scores and the specific recommendations
- Run the bundle analyzer to understand your JavaScript size

Step 2: Fix the biggest issues first

- Optimize images (usually the quickest, highest-impact fix)
- Lazy load routes (reduces initial bundle significantly)
- Remove or replace heavy dependencies

Step 3: Measure again

- Run Lighthouse again after each change
- Compare before/after scores
- Check that you haven't broken anything

Step 4: Set up ongoing monitoring

- Add Vercel Analytics or Google PageSpeed Insights to track real-world performance
- Set performance budgets (e.g., "bundle must stay under 200KB gzipped")
- Check performance after every major feature addition

Prompt for Cursor: "Analyze my React app's performance and suggest specific optimizations. Start by running the bundle analyzer, check for unoptimized images, identify components that should use lazy loading, look for heavy npm packages that have lighter alternatives, and check for unnecessary re-renders. Prioritize fixes by impact, starting with the easiest wins."

Performance Budgets: Setting Limits

A performance budget is a set of limits you commit to not exceeding. Examples:

Metric	Budget
Total JavaScript (gzipped)	< 200 KB
Total page weight	< 1 MB
Lighthouse Performance score	> 90
LCP	< 2.5 seconds
Time to Interactive	< 3.5 seconds
Number of HTTP requests	< 50

You can configure your build tools to warn you (or fail the build) if you exceed these budgets. This prevents performance from slowly degrading as you add features.

Key Takeaways

- Performance directly affects user retention, SEO rankings, and conversion rates. A 1-second delay in page load can reduce conversions by 7%.
- Core Web Vitals (LCP, INP, CLS) are the metrics Google uses to measure real user experience. Aim for green scores on all three.
- Lighthouse is your free, built-in performance auditor. Run it in Incognito mode after every major change. Aim for 90+ on performance.
- Bundle size is the most common performance problem in React apps. Use the bundle visualizer to find heavy dependencies and replace them with lighter alternatives.
- Optimize images first. Converting to WebP, resizing appropriately, and lazy loading below-the-fold images is usually the single highest-impact fix.
- Lazy load routes with `React.lazy()` and `Suspense` to reduce your initial bundle size dramatically.
- Use `React.memo`, `useMemo`, and `useCallback` to prevent unnecessary re-renders, but only where it actually matters (don't optimize prematurely).

- Code splitting breaks your app into smaller chunks that load on demand, so users only download what they need.
- Set performance budgets and monitor them. Performance degrades gradually as features are added, so ongoing vigilance is essential.
- AI tools are excellent at performance optimization. Share your Lighthouse results, bundle analysis, or DevTools screenshots, and they'll suggest specific, actionable fixes.